Solitito — podsumowanie projektu

System rozpoznawania akordów gitarowych w czasie rzeczywistym

Wersja 0.5.8, październik 2026

---

1. Charakterystyka systemu

Solitito jest trainerem gitarowym działającym w czasie rzeczywistym, stworzonym w Rust. Program pobiera sygnał interfejsu audio lub mikrofonu, rozpoznaje wykonywany materiał i prowadzi użytkownika przez standardy jazzowe, interwały, skale, arpeggia, formuły interwałowe oraz orientację na gryfie.

Rozpoznawanie realizuje sieć neuronowa o 7,3 mln parametrów, wyeksportowana do formatu ONNX. Druga, mała sieć o 260 tys. parametrów odpowiada co 16 ms dźwięku na jedno węższe pytanie: która klasa wysokości została właśnie uderzona. Rozstrzyga ona, kiedy dźwięk liczy się ponownie (8.13). Całość przetwarzania — DSP, inferencja oraz interfejs użytkownika — wykonywana jest lokalnie na procesorze, bez połączenia sieciowego i bez usług zewnętrznych.

System udostępnia sześć trybów pracy:

- **Akordy** — pełne standardy jazzowe. Kolor zielony oznacza trafienie dokładne, żółty — triadę lub typowe zastępstwo, czerwony — akord rozpoznany przy sygnale zbyt słabym, by go zatwierdzić.
- **Interwały** — składniki akordu wykonywane pojedynczo, z możliwością wyboru ćwiczonych stopni.
- **Skale** — sekwencyjne przechodzenie dźwięków zgodnie z definicją gamy.
- **Arpeggia** — składniki akordu w sekwencji, na zadanej progresji.
- **Gryf** — losowany jest fragment gryfu obejmujący zestaw strun i cztery progi, po czym zostaje utrzymany; użytkownik proszony jest o kolejne dźwięki leżące w tym obszarze. Tryb służy poznawaniu położenia dźwięków w obrębie jednej pozycji ręki.
- **Formuły** — zbiór interwałów wylosowany nad prymą i grany w dowolnej kolejności, z możliwością postawienia tego samego zbioru na akordzie albo przeniesienia go przez akordy standardu. Tryb opisano w 8.11.

Prace nad projektem rozpoczęto w grudniu 2025 roku. Niniejszy dokument przedstawia architekturę systemu, przebieg prac oraz decyzje projektowe wraz z ich uzasadnieniem.

---

2. Metodyka prac

W projekcie przyjęto zasadę, że każda zmiana wymaga uzasadnienia pomiarowego, a nie hipotezy. W praktyce oznaczało to opracowanie zestawu **sond** — skryptów odpowiadających na pojedyncze pytanie niewielkim kosztem obliczeniowym, bez konieczności ponownego treningu.

sonda	zagadnienie
`verify_annotations.py`	czy etykiety opisują zawartość audio?
`probe_root.py`	jak często pryma wskazana etykietą faktycznie brzmi w oknie?
`probe_sources.py`	która adnotacja akordowa zbioru GuitarSet jest użyteczna?
`probe_quality.py`	z jakiego źródła wyprowadzać jakość akordu?
`inspect_jams.py`	jaka jest rzeczywista zawartość plików JAMS?
`latency_material.py`	wytwarza szarpnięcia o dokładnie znanych momentach ataku, jako wzorzec
`latency_ground_truth.py`	wydobywa ataki i wysokości z PRAWDZIWEGO nagrania, do tego samego pomiaru
`latency_stats.py`	jak późno aplikacja dowiaduje się, co zagrano, i jak często dowiaduje się źle?
`latency_rules.py`	ile kosztowałaby każda z reguł zaliczania na tym nagraniu?

Pierwsze pięć dotyczy danych i modelu; pozostałe cztery tworzą łańcuch i używane są razem. Materiał jest przygotowywany — albo syntetyzowany, z atakami znanymi z konstrukcji, albo pobierany z nagrania rzeczywistej gry — po czym plik przepuszczany jest przez własną ścieżkę cech aplikacji poleceniem `./solitito --probe plik.wav --step 1`, a powstały wydruk czytają dwa ostatnie skrypty. `latency_stats.py` rozdziela trzy odpowiedzi, jakimi aplikacja dysponuje: głowicę wysokości modelu, estymatę jednoramkową oraz — w wydrukach sprzed 0.5.10 — głowicę ataków, którą model miał do tamtej pory. `latency_rules.py` odtwarza na tym samym wydruku reguły zaliczania i podaje dla każdej, ile przyznałaby zaliczeń, których nikt nie zagrał, oraz ile dźwięków pominęłaby zupełnie. Tabela w 8.11 pochodzi z tego skryptu.

Dwa dalsze narzędzia nie są sondami, lecz należą do tego samego zestawu: `gen_weights.py`, wytwarzający rzadkie jądro CQT wspólne dla trenera i aplikacji, oraz `gp5_to_arpeggio.py`, przekładający plik Guitar Pro na zapis stopniami, który czyta tryb Arpeggia. `hf_cleanup.py` czyści repozytorium punktów kontrolnych przed przebiegiem rozpoczynanym od zera. `model_trainer.py` trenuje sieć akordową, a `strike_trainer.py` detektor uderzeń (6.6); `extract_onset_branch.py` wycina detektor uderzeń z połączonego modelu take7.

Regułom zaliczania służą obecnie dwa kolejne narzędzia. Aplikacja potrafi nagrać sesję wraz z dziennikiem każdej podjętej oceny (`SOLITITO_RECORD`), dzięki czemu zaliczenie fałszywe zgłoszone z ćwiczeń można odtworzyć i wyjaśnić. Ponadto harness w zestawie testów skleja 51 zweryfikowanych dźwięków jednego opisanego nagrania w nowe sygnały — dźwięk wybrzmiewający, ten sam dźwięk uderzony ponownie, dźwięk wybrzmiewający w kolejny akord — i przepuszcza przez nie rzeczywisty mechanizm oceny. Z niego pochodzą tabele w 8.13.

Metodyka ta wykazała skuteczność wielokrotnie. Odnotować należy również jej rewers: **hipotezy formułowane przed wykonaniem pomiaru okazywały się błędne w sposób systematyczny.** Zestawienie tych przypadków zawiera rozdział 9.

3. Zbiór syntetyczny

Skrypt `dataset_generator_v2.py` wytwarza w jednym przebiegu plik Guitar Pro **oraz** komplet adnotacji.

3.1. Zawartość

394 bloki o czasie trwania 6 sekund (3 takty przy 120 BPM), obejmujące:

- 12 prym × {maj, min, maj7, dom7, min7, m7b5, dim7, sus4, aug} w kilku pozycjach na gryfie,
- wszystkie 96 pojedynczych dźwięków (6 strun × 16 progów).

Struktura bloku: takt pierwszy stanowi atak, drugi — wybrzmienie (tie), trzeci — ciszę. Adnotacja obejmuje przedział `[start + 0,05 s, +3,2 s]`, to jest atak wraz z sustainem, z pominięciem ogona zaniku.

3.2. Weryfikacja akordów

Przed rozpoczęciem generacji skrypt sprawdza **każdy z 21 ruchomych akordów na każdym progu**, weryfikując, czy akord realnie daje deklarowane interwały. Błąd w tabeli akordów zatrzymuje generację zamiast propagować się do zbioru danych.

3.3. Render

Ścieżka gitary jest eksportowana jako sygnał DI i renderowana w środowisku DAW przy użyciu NAM w dwóch wariantach:

- `synth_dataset_clean.wav` — Fender Deluxe Reverb, brzmienie czyste,
- `synth_dataset_eob.wav` — na granicy przesteru.

Zalecaną częstotliwością próbkowania jest **48 kHz**, z dwóch niezależnych powodów. NAM pracuje natywnie w 48 kHz, wobec czego przy 44,1 kHz wtyczka wykonuje resampling wewnętrzny. Ponadto decymacja do 16 kHz, na których pracuje model, jest przy 48 kHz dokładna (`48000/16000 = 3`), a przy 44,1 kHz — nie (`2,75625`).

3.4. Kalibracja i weryfikacja

Tryb `--calibrate <wav>` wyznacza moment pierwszego ataku na wypadek, gdyby środowisko DAW dodało ciszę na początku pliku. Obsługiwane są formaty PCM 16/24/32-bit oraz zmiennoprzecinkowe 32/64-bit, mono i stereo — przy użyciu własnego czytnika WAV, bez zależności zewnętrznych.

Skrypt `verify_annotations.py` porównuje etykietę z **faktyczną zawartością audio**: dominującą klasą wysokości w oknie zestawia się z prymą wskazaną etykietą. Jedyną zależnością jest numpy.

Wartości odniesienia uzyskane z renderu v2:

	clean	eob	wartość losowa
top1	87%	77%	~8%
top3	**100%**	**98%**	~25%

Miarodajna jest wartość top3, ponieważ w akordzie pryma bywa cichsza od tercji lub kwinty. Wartość `top3 > 75%` kwalifikuje zbiór do treningu; `top3 ≈ 25%` oznacza, że etykiety nie opisują sygnału.

Skrypt weryfikuje ponadto okres bloków wyznaczony z obwiedni energii oraz przesunięcie pomiędzy początkiem adnotacji a atakiem. Oba testy są **komplementarne**: przesunięcie adnotacji o całkowitą wielokrotność bloku trafia w atak sąsiedniego bloku i pozostaje niewykrywalne dla testu czasowego. Wykrywa je dopiero porównanie etykiet z zawartością sygnału.

Przyjęto zasadę nadrzędną: **generator wypisuje etykiety wprost, a niezależny skrypt weryfikuje ich zgodność z audio.** Żaden krok przetwarzania nie odtwarza etykiet z sygnału.

---

4. Zbiór GuitarSet

GuitarSet obejmuje 360 nagrań z adnotacjami w formacie JAMS, zarejestrowanych przetwornikiem heksafonicznym. Jest to jedyne źródło materiału z rzeczywistego instrumentu wykorzystane w projekcie. Doprowadzenie go do postaci użytecznej wymagało czterech przebiegów treningowych.

Poniżej opisano cztery właściwości zbioru, których pominięcie obniża dokładność modelu.

4.1. Połowa zbioru nie zawiera akordów

Każdy fragment zarejestrowano dwukrotnie: jako `_comp` (akompaniament) oraz `_solo` (improwizacja jednogłosowa). **Adnotacja akordowa jest w obu przypadkach identyczna** — opisuje progresję, nad którą wykonawca improwizował.

Trening głowic akordowych na plikach solo sprowadza się do uczenia modelu, że pojedynczy dźwięk stanowi pełny akord jazzowy. Materiał solowy obejmuje 180 z 360 plików.

Odfiltrowanie tych plików przesunęło wskaźnik `Exact` z **44,8% na 82,3%** w jednym przebiegu.

Cele **pitch** pochodzące z plików solo pozostają w pełni poprawne — jest to rzeczywiste wykonanie jednogłosowe z dokładnymi adnotacjami nutowymi, a więc materiał odpowiedni dla detektora pojedynczych dźwięków. Trener zachowuje zatem stratę pitch na oknach solowych i maskuje wyłącznie prymę oraz jakość (`GUITARSET_SOLO_MODE = "mask_chord"`).

Odnotować należy konsekwencję pośrednią. Sonda `probe_root.py` wyznaczała początkowo słyszalność prymy na wszystkich 360 plikach, uzyskując sufit 64,1%. Na tej podstawie sformułowano wniosek, że wykonawcy jazzowi stosują voicingi bez prymy, a nazwa akordu nie jest funkcją sygnału. Wniosek ten okazał się nieprawidłowy: po odfiltrowaniu materiału solowego pryma brzmi w **97%** okien akompaniamentu.

4.2. Adnotacje akordowe występują w dwóch wariantach

Każdy plik zawiera adnotację `instructed` (akord wynikający z zapisu) oraz `performed` (transkrypcja wykonania). Liczba segmentów jest identyczna, rozkład etykiet — różny:

jakość	instructed	performed	różnica
maj	2640	2106	−534
min	960	460	**−500**
min7	**0**	**360**	**+360**
maj7	**0**	**430**	**+430**
dom7	480	694	+214
m7b5	240	134	−106
sus	0	132	+132
razem	**4320**	**4320**	**0**

Suma segmentów pozostaje niezmieniona, wobec czego nie jest to wybór pomiędzy większą a mniejszą liczbą danych, lecz **przeetykietowanie tych samych nagrań**.

Wiersze `min` oraz `min7` należy rozpatrywać łącznie: **pięćset segmentów oznaczonych w zapisie jako `m` zostało wykonanych jako `m7`.** Trening na adnotacji `instructed` uczy model, aby voicing zawierający septymę małą klasyfikować jako zwykły akord molowy. Błąd ten był następnie obserwowany w aplikacji, gdzie akord `Gm7` rozpoznawano jako `Gm`.

Ponadto adnotacja `instructed` nie zawiera **ani jednego** wystąpienia klas `maj7` i `min7`. Do momentu przełączenia obie klasy pochodziły wyłącznie z dwóch renderów zbioru syntetycznego, to jest z jednego instrumentu przetworzonego przez jeden wzmacniacz. Skutkowało to wartością 100% na zbiorze walidacyjnym (ten sam instrument po obu stronach podziału) przy jednoczesnym braku odporności na materiał rzeczywisty.

Przełączenie na adnotację `performed` przesunęło wskaźnik `Exact` z **82,3% na 92,4%**. Dokładność rozpoznawania prymy pozostała bez zmian: obie adnotacje różnią się co do prymy w **0 z 43 056** porównań.

4.3. Podział zbioru musi przebiegać po źródle

Losowe potasowanie listy segmentów akordowych i podział w proporcji 94/6 umieszcza sąsiadujące takty **tego samego nagrania** po obu stronach podziału — przy identycznym instrumencie, pomieszczeniu, mikrofonie i ujęciu, często przy tym samym akordzie występującym takt później.

W zbiorze syntetycznym zależność jest silniejsza: rendery `clean` i `eob` jednego bloku stanowią to samo wykonanie przetworzone przez inny wzmacniacz, a trafiały do zbioru treningowego i walidacyjnego niezależnie.

Zastosowane rozwiązanie polega na grupowaniu po źródle: całym pliku dla zbioru GuitarSet oraz całym bloku (obu renderach) dla zbioru syntetycznego.

Wszystkie wskaźniki walidacyjne ulegają po tej zmianie obniżeniu. Nie stanowi to regresji modelu, lecz usunięcie zawyżenia, które unieważniało wcześniejsze wnioski dotyczące generalizacji. Wartość `root_acc = 98%` uzyskana w przebiegu take1 była w znacznej mierze artefaktem: przy adnotacjach `both` ten sam segment występował dwukrotnie, wobec czego identyczne okno trafiało zarówno do zbioru treningowego, jak i walidacyjnego.

4.4. Cele pitch wyznaczane z `note_midi`

Adnotacja akordowa opisuje **zamierzoną harmonię** w skali wielu sekund. Okno treningowe obejmuje 0,77 s i często nie zawiera etykietowanej septymy. Model był zatem karany za nieprzewidzenie dźwięku nieobecnego w sygnale — recall septym na zbiorze GuitarSet wynosił 32%.

Przetwornik heksafoniczny dostarcza adnotacji `note_midi`, opisujących rzeczywiste wykonanie na każdej strunie osobno. Wyznaczenie celów pitch na ich podstawie podniosło recall septym z **32% do 96%**.

Przyjęty próg: dźwięk musi brzmieć przez co najmniej 25% okna (`NOTE_MIN_COVER`), aby zostać uwzględniony w celu.

---

5. Architektura

5.1. Ścieżka sygnału

> `wejście audio → resampling do 16 kHz → FFT (8192) → rzadkie pseudo-CQT → cechy → model ONNX`

Resampling do 16 kHz. Transformata CQT obejmuje 6 oktaw począwszy od C1, wobec czego najwyższy bin przypada w okolicy 2 kHz, a więc znacznie poniżej granicy Nyquista wynoszącej 8 kHz. Pasmo nie stanowi ograniczenia.

Pseudo-CQT. Zamiast właściwej transformaty o stałym współczynniku Q aplikacja mnoży widmo FFT przez wyznaczone uprzednio jądro: 144 biny, 24 na oktawę, co odpowiada rozdzielczości ćwierćtonowej. Jądro pochodzi z funkcji `librosa.filters.constant_q`, dzięki czemu aplikacja i trener wytwarzają identyczne cechy.

Cechy. 168 wartości na ramkę:

zakres	zawartość
0–143	biny CQT po log-normalizacji
144–155	chroma (macierz `cq_to_chroma`, normalizacja maksimum w ramce)
156–167	energia basowa (średnia z par binów 0–23)

Model obejmuje 48 ramek historii przy skoku 256 próbek, co odpowiada **0,77 s**.

5.2. Sieć

```
wejście [48, 168]
   ↓
InstanceNorm2d
   ↓
ConvBlockSE  1 → 48       (Squeeze-and-Excitation)
ConvBlockSE 48 → 96
ConvBlockSE 96 → 192
ConvBlockSE 192 → 384
   ↓
Linear 3840 → 384
   ↓
+ token CLS, kodowanie pozycji
   ↓
TransformerEncoder: 4 warstwy, 8 głowic, d=384, FF=768, GELU, norm_first
   ↓
CLS
   ├── fc_root     → 13   (12 klas wysokości + „Noise")
   ├── fc_quality  → 11   (maj, min, maj7, dom7, min7, m7b5, dim7, aug, sus, note, N)
   └── fc_pitch    → 12   (sigmoid: które klasy wysokości brzmią)
```

Łączna liczba parametrów: 7 286 038. To `best_model_v2_take6_chords.onnx`, plik, który ładuje aplikacja.

Do wersji 0.5.10 sieć miała czwartą głowicę, `fc_onset` (156 156 parametrów, trenowaną w fazie 4, 6.1): które klasy wysokości zostały UDERZONE. Nie czytała tokenu CLS. Jej wejście składało się z czterech części — ostatniej ramki enkodera, różnicy między nią a ramką sprzed sześciu ramek oraz, pobranych z surowych cech, zanim zobaczy je enkoder, PRZYROSTU CQT złożonego na kla-

sy wysokości wraz z przyrostem chromy: atak dokłada energii do widma, a wybrzmiewanie nie. Detektor uderzeń (5.4) odpowiada na to samo pytanie lepiej i zastąpił ją w sędzim wszędzie; plik ładowany przez aplikację eksportowany jest bez tej głowicy, a jego trzy wyjścia są co do bitu takie jak w pliku czterogłowicowym.

5.3. Podział zadań pomiędzy głowicami

Rozróżnienie ról poszczególnych głowic ma charakter kluczowy i zostało potwierdzone pomiarem.

głowica	wynik	rola
`pitch_logits`	F1 0,909	które dźwięki brzmią — podstawa trybów Interwały, Skale, Arpeggia i Gryf
`root_logits`	98,1%	nazwa prymy
`quality_logits`	~93%	rodzina akordu

Wczesna wersja aplikacji wyprowadzała jakość akordu z wektora pitch przy użyciu progów ustalonych ręcznie. Sonda `probe_quality.py` zestawiła trzy metody na tym samym checkpoincie:

metoda	dokładność
głowica `quality_logits`	**80,5%**
dopasowanie szablonów do przewidzianego pitch	66,0%
dopasowanie szablonów do **rzeczywistego** wektora pitch	59,2%

Głowica przewyższa dopasowanie szablonów do *dokładnie znanego* zbioru dźwięków o 21 punktów procentowych. Wyprowadza zatem z sygnału informację nieobecną w samym zbiorze klas wysokości: barwę, rozłożenie voicingu w rejestrze oraz kształt ataku.

Wniosek projektowy: głowica jakości pozostaje elementem koniecznym.

5.4. Detektor uderzeń

Żadna z trzech głowic nie mówi, co zostało UDERZONE. Dawna głowica ataków dzieliła z modelem rozpoznawania jego okno i rytm: 0,77 s dźwięku, pytanie co 40 ms. To zbyt zgrubnie, by rozstrzygnąć, czy dźwięk już brzmiący został właśnie uderzony ponownie — a to jest pytanie, które stawiają dźwięki powtórzone. Detektor uderzeń jest osobną siecią zbudowaną wyłącznie dla niego:

```
dwa okna Hanna najnowszego dźwięku, 1024 i 2048 próbek (64 i 128 ms)
   ↓  amplitudy do 4 kHz, log1p(1000·a)/log(1001): 770 cech na krok 16 ms
Conv1d 770 → 96  +  Conv1d 770 → 96 na PRZYROŚCIE każdego binu względem czterech ramek
wcześniejszych
   ↓
4 × przyczynowy rezydualny Conv1d 96 → 96, jądro 3, dylatacje 1, 2, 4, 8
   ↓
Conv1d 96 → 12   (sigmoida: które klasy wysokości uderzono w tej ramce)
```

Sieć jest przyczynowa i potrzebuje 34 ramek przeszłych oraz bieżącej — 0,56 s. Ma 260 076 parametrów i kosztuje 0,7 ms na krok na procesorze, więc aplikacja pyta ją w każdym kroku. Rzut przyrostu niesie tę samą myśl co wejście dawnej głowicy ataków: atak dodaje widmu energii, a wybrzmiewanie nie.

Została wytrenowana jako gałąź modelu połączonego, obok zamrożonej sieci akordowej (6.6). Aplikacja ładuje ją wyciętą z tego pliku, `short_onset_masking_v2.onnx` (1 MB), ponieważ gałąź nigdy nie czyta wejścia akordowego: jej odpowiedź jest identyczna co do ostatniej cyfry niezależnie od jego zawartości.

---

6. Trening

6.1. Fazy

Procedura treningowa obejmuje cztery fazy, przy czym zasadnicze znaczenie ma faza pierwsza.

faza	zakres	status
1	trening zasadniczy, 120 epok, cosine LR z rozgrzewką	jedyna wnosząca poprawę
2	strojenie progu głowicy pitch	skorygowana — sortowała po metryce niezależnej od progu
3	dostrajanie głowic przy zamrożonym enkoderze	**wyłączona**
4	głowica ataków, trenowana osobno	trenowana, już niewydawana: od 0.5.10 eksport ją pomija

Faza 4 trenuje `fc_onset` przy zamrożonych wszystkich pozostałych parametrach. Konstrukcja jest celowa: faza albo daje głowicę wartą użycia, albo zostawia model dokładnie takim, jakim był, a trzy wcześniejsze wyjścia nie mogą przesunąć się o miejsce po przecinku. Materiałem jest adnotacja na poziomie dźwięku — okna próbkowane wokół rzeczywistych ataków, domyślnie wyłącznie z nagrań solowych, bo to one niosą `note_midi`. Trening kończy przegląd progów raportowany jako F1. Głowicy nie ma w pliku, który ładuje aplikacja: po fazie 4 trener eksportuje sieć akordową bez niej, `best_model_v2_take6_chords.onnx`, a o tym, co uderzone, sędzia dowiaduje się od detektora uderzeń (5.4).

Faza 2 skanowała progi w zakresie 0,30–0,70, optymalizując wskaźnik `exact`. Wskaźnik ten stanowi koniunkcję `argmax(root)` oraz `argmax(quality)`, wobec czego przyjmował identyczną wartość dla wszystkich 41 progów, a wybór był losowy. Obecnie sortowanie odbywa się po F1 głowicy pitch, na którą próg faktycznie oddziałuje.

Fazę 3 wyłączono po zmierzeniu jej efektów w trzech kolejnych przebiegach:

```
take2, 40 epok:  pitch_f1 0,9318 → 0,9326 (+0,0008), exact 0,5455 → 0,5445
take3,  4 epoki: F1 0,933 → 0,931,             exact 54,6% bez zmian
```

Enkoder pozostaje zamrożony, uczeniu podlegają wyłącznie głowice przy współczynniku uczenia 1e-5. Faza nie dysponuje mechanizmem poprawy modelu, a jej koszt wynosi około 1,5 godziny obliczeń.

6.2. Funkcje straty i maskowanie

- **root** — CrossEntropy z wygładzaniem etykiet 0,05,
- **quality** — CrossEntropy z wygładzaniem, sampler ważony po klasie,
- **pitch** — Focal BCE (γ = 2,0, `pos_weight` 2,5), waga pomocnicza 0,7.

Zastosowano dwa mechanizmy maskowania, oba uzasadnione pomiarem.

`MASK_ROOT_WHEN_SILENT` — strata prymy wyznaczana wyłącznie na oknach, w których pryma faktycznie brzmi. Trening prymy na oknach jej pozbawionych nie prowadzi do wyuczenia percepcji, lecz do zapamiętania progresji zbioru GuitarSet, przy czym wspólny enkoder otrzymuje gradient sprzeczny z celem pitch.

`GUITARSET_SOLO_MODE = "mask_chord"` — pryma i jakość nie otrzymują gradientu z nagrań solowych; głowica pitch otrzymuje go bez zmian.

6.3. Augmentacja

- **przesunięcie wysokości** o ±N półtonów. Istotny szczegół implementacyjny: CQT oraz energia basowa przesuwane są **z wypełnieniem zerami**, chroma — **cyklicznie**. Chroma jest z definicji okrężna, CQT nie jest; zawinięcie pasma basowego na górę zakresu wprowadzałoby dźwięki nieobecne w sygnale.
- **maskowanie czasu i częstotliwości** (SpecAugment),
- **nachylenie widma oraz szum** — symulacja zróżnicowanych torów sygnału.

6.4. Bramka energetyczna

Parametr `ENERGY_KEEP_FRAC = 0,55` odrzuca okna o energii poniżej 55% wartości szczytowej segmentu. Uzasadnienie: w fazie zaniku septyma, będąca najcichszym składnikiem voicingu, zanika jako pierwsza, podczas gdy etykieta pozostaje niezmieniona. Brak bramki prowadziłby do systematycznego uczenia kolapsu `m7 → m`.

6.5. Metryki

Metryki akordowe wyznaczane są **wyłącznie na oknach, w których etykieta opisuje sygnał**, z pominięciem okien solowych. Raportowane są dodatkowo w rozbiciu na okna ze słyszalną prymą i bez niej, ponieważ wartość łączna miesza dwie odmienne populacje.

Wybór najlepszego checkpointu odbywa się według wskaźnika `composite = (root_audible + qual + exact) / 3`. Zastosowanie łącznego `root_acc` premiowałoby model skutecznie odtwarzający progresje zamiast modelu poprawnie analizującego sygnał.

Kontrola diagnostyczna `TRAIN`, wykonywana co 5 epok, wyznacza metryki na danych treningowych bez augmentacji. Odpowiada na pytanie, czy model jest w stanie odwzorować własne dane treningowe. Odpowiedź negatywna wskazuje na cechy lub etykiety jako źródło ograniczenia, nie na generalizację, i oznacza, że zwiększanie liczby epok jest bezcelowe.

6.6. Detektor uderzeń

Detektor uderzeń trenowany jest osobno od sieci akordowej, własnym trenerem: obie sieci nie dzielą ani wag, ani wejścia, tylko aplikację. Jego materiał jest opisany atakami, nie akordami:

- **GuitarSet**, całe nagrania solo i comp, z graczami dzielonymi w całości — 00–03 do treningu, 04 do walidacji, 05 do testu. Początki dźwięków pochodzą z adnotacji, nie są potwierdzonymi atakami kostki.
- **Syntetyczne szarpnięcia** (Karplus–Strong), których czasy ataku są dokładne z konstrukcji: dźwięki trzymane, ten sam dźwięk szarpnięty ponownie w trakcie wybrzmiewania, oktawy, tercja lub kwinta dodana nad brzmiącą prymą, ponownie uderzone trójdźwięki.

- **Pary maskujące**: pryma i tercja wybrzmiewają, po czym pada górna kwinta, −18, −12, −6 lub 0 dB względem nich, mierzone w jej pierwszych 96 ms. Każda występuje w trzech wariantach o wspólnym tle i wzmocnieniu — samo tło, sama kwinta, oba razem — tak że jedyną różnicą między nimi jest uderzenie.

Poziom w treningu zmienia się o ±6 dB. Epokę wybiera się na walidacji przy stałym progu 0,5, próg następnie na wyeksportowanym ONNX — tym, co aplikacja faktycznie uruchomi — a zbiór testowy ocenia się raz, przy tym progu. Wydany model powstał w trzech przebiegach, każdy douczany od najlepszych wag poprzedniego: Rise od zera, następnie z zamrożoną bazą akordową (`v2_take7`), następnie z parami maskującymi (`v2_take7_masking_v2`, próg 0,9).

Trener detektora uderzeń jest jednym plikiem, `dist/strike_trainer.py`, który można wkleić w całości do notebooka. Zastąpił kilkanaście skryptów eksperymentalnych i trzy generatory, które je składały; same eksperymenty pozostają w gałęzi `rise`. Scalenie zweryfikowano, a nie założono: na procesorze jeden plik trenuje bit w bit tak samo jak skrypty, którymi wytrenowano wydany model — te same dane, partie, straty, wagi, próg i wyjścia ONNX, zarówno od zera, jak i od punktu kontrolnego rodzica — a ponowny przebieg wydanego przepisu na GPU Kaggle doszedł do tych samych wag. Plik dla aplikacji zapisuje wprost z wytrenowanej sieci; z wydanego punktu kontrolnego odpowiada on bit w bit tak samo jak wydany `short_onset_masking_v2.onnx`. Sieć akordowa ma własny trener, `dist/model_trainer.py`, ten, który wytworzył `best_model_v2_take6_onset.onnx`: wyeksportowany z punktu kontrolnego tamtego przebiegu daje na wszystkich czterech wyjściach to samo co wydany plik, bit w bit. Zapisuje też trójwyjściowy plik ładowany przez aplikację, `best_model_v2_take6_chords.onnx`, którego wyjścia są co do bitu trzema pierwszymi wyjściami pliku czterowyjściowego. 770 cech liczonych jest dwukrotnie, przez trener w Pythonie i przez aplikację w Ruście; wspólny plik testowy utrzymuje obie strony przy tych samych liczbach, bit w bit, w obu zestawach testów.

---

7. Wyniki

Model `v2_take6`, walidacja z podziałem po źródle, z pominięciem okien solowych:

metryka	wartość
dokładność prymy	**98,1%**
pitch F1	**0,909**
trafienie dokładne (pryma **i** jakość)	**92,4%**
F1 ataków (głowica już niewydawana)	**0,812**

Trzy pierwsze wielkości są identyczne w pliku trójgłowicowym i czterogłowicowym: głowica ataków trenowała się przy zamrożonej reszcie sieci. Czwarta podana jest przy progu maksymalizującym F1 na zbiorze walidacyjnym; od 0.5.10 głowicy nie ma w wydanym modelu.

Dokładność w podziale na jakości przy najlepszym checkpoincie: `dom7` 97%, `min7` 93%, `min` 92%, `sus` 91%, `maj` 89%, `maj7` 89%; klasy `m7b5`, `dim7` oraz `aug` powyżej 97%.

7.1. Przebieg prac

przebieg	zmiana	Exact
take1–take3	różne, przy podziale z przeciekiem	nieporównywalne
take4	podział po źródle — punkt odniesienia	44,8%
take5	maskowanie nagrań solowych	82,3%
take6	adnotacje `performed`	**92,4%**

7.2. Ograniczenie dokładności

Różnica pomiędzy zbiorem treningowym a walidacyjnym w zakresie jakości wynosi **6,5 punktu procentowego** (99,2% wobec 92,7%). Model odwzorowuje dane treningowe. Odpowiada to profilowi ograniczenia przez **generalizację**, nie przez pojemność architektury.

Wniosek praktyczny: zwiększenie liczby epok ani rozmiaru modelu nie przyniesie poprawy. Poprawę przyniesie zwiększenie ilości zróżnicowanego materiału z rzeczywistego instrumentu.

---

8. Aplikacja

Dokładność rozpoznawania nie jest równoznaczna z użytecznością trenażera. Trzy zagadnienia okazały się mieć wagę porównywalną ze zmianami w modelu.

8.1. Wymóg pełnego okna kontekstowego

Trener wyznaczał okna **wyłącznie wewnątrz** wybrzmiewającego akordu (`range(start, koniec - 48)`). Po uderzeniu w struny bufor aplikacji przez 0,77 s zawiera częściowo ciszę; uwzględniając okno FFT (8192 próbki, czyli 512 ms), najstarsza ramka opisuje sygnał sprzed nawet 1,3 s. Stanowi to wejście spoza rozkładu treningowego.

Zaobserwowany objaw: akordy septymowe rozpoznawane były dopiero w fazie wybrzmiewania, to jest w pierwszym momencie, w którym okno zostaje w całości wypełnione akordem.

Zastosowanym rozwiązaniem był początkowo jeden próg: aplikacja nie kierowała zapytania do modelu, dopóki okno nie było wypełnione sygnałem w 90%. Próg ten jest właściwy dla NAZWY akordu i niewłaściwy dla wszystkiego pozostałego. Granie po jednym dźwięku nigdy nie wypełnia okna w dziewięciu dziesiątych — to 43 ramki, czyli 688 ms nieprzerwanego dźwięku — wobec czego model nie był pytany wcale, ekran zastygał na ostatnim akordzie, a wraz z nim zastygało wszystko, co z niego korzysta.

Wymaganie jest obecnie rozdzielone na dwa. Model pytany jest od połowy okna (pomiar: przy wypełnieniu 50–70% głowica wysokości nazywa odosobniony dźwięk poprawnie w każdej ramce pomiaru), a jego nazwie akordu wierzy się dopiero od dziewięciu dziesiątych. Oba progi zapisane są jako stałe w jednym miejscu aplikacji, ponieważ bramka na nazwie stosowana jest po drugiej stronie kanału niż wątek pytający — zapisane dwukrotnie mogłyby się rozjechać.

8.2. Zróżnicowany czas wybrzmiewania składników akordu

Podgląd diagnostyczny przytrzymanego akordu `Gm7`:

```
G m7 | min7=96% | b7=96      ← bezpośrednio po uderzeniu
G m7 | min7=82% | b7=76
G m7 | min7=52% | b7=52
G m  | min=49%  | b7=45      ← septyma wyciszona, model zmienia klasyfikację
```

Klasyfikacja modelu jest poprawna — w bieżącym oknie septyma faktycznie nie występuje. Akord nie zmienia jednak tożsamości w trakcie wybrzmiewania.

Zastosowane rozwiązanie: zatrzask jakości. Mechanizm **załącza się** przy pewności $\geq 0{,}60$, natomiast **utrzymuje stan niezależnie od niej**. W fazie zaniku model raportuje uboższą jakość z pewnością rzędu 94–96%, wobec czego sam próg pewności nie stanowiłby zabezpieczenia. Zwolnienie zatrzasku następuje przy nowym ataku lub przy zmianie prymy.

Zatrzask załącza się dopiero po 48 klatkach od ataku. Wcześniej okno zawiera jeszcze ogon **poprzedniego** akordu, co skutkowałoby zatrzaśnięciem nieprawidłowej nazwy.

Detekcja ataku porównuje poziom sygnału z wolnozmienną obwiednią (EMA), a nie z progiem bezwzględnym, który zależałby od głośności wykonania. Zastosowano ponadto refrakcję 0,2 s, aby pojedyncze szarpnięcie wyzwalało dokładnie jeden atak.

8.3. Pomiar czasu zamiast wartości założonej

Licznik postępu otrzymywał wartość stałą `dt = 0,040`, podczas gdy wątek inferencji wymagał 55–90 ms na cykl (inferencja wraz z 40 ms uśpienia). Licznik pracował zatem wolniej od zegara rzeczywistego: próg 0,6 s osiągany był po około sekundzie, przy czym wartość zależała od obciążenia maszyny.

Po korekcie, obejmującej pomiar czasu rzeczywistego, wyznaczanie okresu przez wątek inferencji od początku cyklu, zawężenie okna głosowania z 5 do 3 oraz obniżenie domyślnego progu do 0,25 s, przejście trwa **około 0,3 s** zamiast 1,2 s.

8.4. Rzadka reprezentacja jądra CQT

Pełne jądro obejmuje 4097 × 144 = 589 968 wag, skoncentrowanych wokół częstotliwości środkowej każdego binu. Odrzucenie wag poniżej 1e-4 wartości szczytowej daje następujące wyniki:

próg	zachowanych wag	maks. błąd względem szczytu
1e-5	21,9%	0,006%
1e-4	**6,9%**	**0,033%**
1e-3	2,3%	0,352%

Błąd wyznaczono na trzech widmach: białym, różowym oraz harmonicznej serii gitarowej. Po transformacie CQT następuje log-normalizacja do zakresu 80 dB, wobec czego wartość 0,03% pozostaje o rzędy wielkości poniżej rozdzielczości cechy.

Rozmiar pliku wag zmniejsza się z **28 MB do 2 MB**, a ścieżka audio wykonuje około **14-krotnie mniej mnożeń** na ramkę. Poprzednia implementacja przechodziła wszystkie 4097 binów FFT dla każdego ze 144 binów CQT, odsiewając wartości zerowe dopiero wewnątrz pętli.

8.5. Zgodność cech pomiędzy trenerem a aplikacją

Rozbieżności tej klasy są szczególnie trudne w diagnozie, ponieważ aplikacja pozostaje funkcjonalna, wykazując jedynie błędy klasyfikacji. Zidentyfikowano dwie:

- **mapowanie chromy.** Plik dystrybuowany z aplikacją zwijał biny parami `(0,1), (2,3), …`, natomiast `librosa.cq_to_chroma` stosuje podział `(1,2), (3,4), …`. Co drugi bin trafiał do klasy sąsiedniej, co odpowiada rozmyciu chromy o pół tonu na połowie pasma.
- **klucz pamięci podręcznej.** Nazwa pliku cache pochodziła z wyrażenia `abs(hash(ścieżka))`. Interpreter Pythona losuje ziarno funkcji skrótu dla łańcuchów przy każdym uruchomieniu procesu, wobec czego pamięć podręczna nie była wykorzystywana pomiędzy sesjami. Obecnie stosowany jest skrót SHA-1 z nazwy pliku.

Aplikacja **odrzuca** wagi w poprzednim, gęstym formacie, sygnalizując to komunikatem.

8.6. Interfejs użytkownika

Po przeglądzie liczba regulatorów została zredukowana z pięciu do czterech:

regulator	uwagi
Bramka szumu	w dBFS, z miernikiem poziomu w tej samej skali i znacznikiem progu
Pewność akordu	próg dla nazwy akordu (tryb Akordy)

regulator	uwagi
Próg dźwięku	próg dla pojedynczego dźwięku (tryby dźwiękowe)
Czas przytrzymania	wymagany czas utrzymania poprawnego akordu

Usunięto regulatory `Tail` (ustawiany z interfejsu i nieodczytywany w żadnym miejscu kodu) oraz `In gain` (którego wpływ znosiła normalizacja wykonywana w obrębie ramki, wobec czego przesuwał on wyłącznie tę samą nierówność co bramka szumu).

Regulator `Confidence` sterował dwiema różnymi wielkościami jednocześnie, a w trybach dźwiękowych podlegał ograniczeniu dolnemu `.max(0.5)`, w wyniku czego cały zakres 0,1–0,5 dawał identyczne zachowanie. Funkcje rozdzielono.

Bramka szumu operowała uprzednio w liniowej skali RMS 0–0,1, która **nie obejmowała poziomu szumu mikrofonu laptopowego** (RMS 0,05–0,15 po wzmocnieniu). Skala decybelowa −72… 0 dBFS zapewnia rozdzielczość w wymaganym zakresie oraz zasięg do pełnej skali.

Panel przerósł od tego czasu pojemność jednej kolumny i podzielony jest na cztery zakładki — wejście wraz z bramką, surowość oceny, materiał do zagrania oraz zawartość okna. Trzecia z nich zawiera wyłącznie to, co należy do trybu widocznego na ekranie: utwór nie ma nic do powiedzenia w Formułach, a formuła nic w Akordach, więc w każdym trybie jest to inna zakładka. Rysowana jest zawsze jedna zakładka, wobec czego odświeżaniu w trakcie gry podlega odpowiednio mniej.

8.7. Tryb diagnostyczny

```
SOLITITO_DEBUG=1 ./solitito
```

Tryb wypisuje przy każdej predykcji trzy najsilniejsze jakości oraz wektor pitch przeliczony na **interwały względem rozpoznanej prymy**:

```
G m7  | min7=97% sus=0% maj=0% | R96# b25 28 b382# 37 44 b56 594# b616 69 b797# 74
```

Narzędzie rozróżnia przypadek, w którym model nie wykrywa septymy, od przypadku, w którym ją wykrywa, lecz pomija w klasyfikacji. Oba objawy są nierozróżnialne na poziomie nazwy akordu i prowadzą do przeciwnych działań korygujących. Przypadek `Gm7` rozstrzygnięto przy jego użyciu bez ponownego treningu.

8.8. Pojedynczy dźwięk nie jest pytaniem, na które model potrafi odpowiedzieć

Model pytany jest o 48 ramek — 0,77 s — i odpowiada o całości tego materiału. Jest to właściwe dla akordu trzymanego pod palcami i niewłaściwe dla gamy, w której dźwięki następują po sobie szybciej, niż okno zdąży się opróżnić.

Pomiar narzędziem `--probe` na gamie granej po 0,6 s na dźwięk, przy obowiązującej wówczas regule (klasa docelowa powyżej progu i w granicach 10% od najgłośniejszej):

	dźwięk aktualnie grany	dźwięk poprzedni
głowica wysokości modelu	7% okien	79%
pojedyncza ramka CQT	57%	43%

Wina nie leży po stronie modelu: na dźwiękach izolowanych i trzymanych przypisuje on 0,96–0,99 właściwej klasie, a na gamie raportuje oba dźwięki, ponieważ oba znajdowały się w oknie. Starszy z nich wygrywa poziomem, mając za sobą większą część okna.

Przyjęte rozwiązanie: tryby nutowe zadają drugie pytanie pojedynczej ramce CQT, pozbawionej pamięci. Suma harmoniczna po logarytmicznych prążkach — wobec logarytmicznej osi jest to widmo iloczynu harmonicznych — wskazuje klasę wysokości brzmiącą w danej chwili. Ani razu nie wskazała klasy, która nie została zagrana.

Domyślnie oszacowanie to wyłącznie **dokłada** drogę do zaliczenia, ponieważ oddanie mu rozstrzygnięcia kosztowałoby własność odróżniającą ten trenażer od monofonicznego: głowica wysokości jest polifoniczna, więc akord zagrany jednym pociągnięciem zalicza swoje interwały po kolei. Opcja **Graj dźwięki pojedynczo** czyni oszacowanie rozstrzygającym i dodatkowo wymaga nowego ataku, zanim powtórzony dźwięk zostanie zaliczony po raz drugi. Który atak się liczy, opisuje 8.13.

Pozostałe opóźnienie wnosi okno FFT o długości 8192 próbek, czyli pół sekundy, i to ono sprawia, że dźwięki krótsze niż około 0,4 s pozostają trudne. Estymator o krótszym oknie, działający w dziedzinie czasu (autokorelacja), jest drogą, która pozostaje otwarta.

8.9. Wybór wejścia i to, czego lista urządzeń nie pokazuje

Maszyna z systemem Windows nie podawała sygnału do momentu ręcznej zmiany częstotliwości próbkowania, co wykazało, że format próbki zwracany przez backend nie może być pomijany, a wybór urządzenia należy do użytkownika, nie do domyślnej konfiguracji systemu.

Lista urządzeń ma jedną własność nieoczywistą: **kartę można otworzyć raz.** Cokolwiek ją trzyma — serwer dźwięku, inna aplikacja albo własny strumień tego programu — usuwa ją z wyliczenia całkowicie. Wynikają z tego trzy konsekwencje, z których każda została najpierw zaobserwowana jako usterka:

- lista zbudowana po otwarciu strumienia nie zawiera karty, z której trwa nagrywanie,
- pod PipeWire, który przejmuje sprzęt, pozostają wyłącznie cztery nazwy serwerowe,
- urządzenie, z którego nagrywamy, musi być wyłączone spod oznaczenia „niedostępne", ponieważ jego nieobecność w skanie jest właśnie dowodem, że działa.

Bramka szumu zapamiętywana jest per urządzenie. Interfejs i mikrofon laptopa dzielą dziesiątki decybeli, a próg, który trzeba odnajdywać po każdym przełączeniu, nie jest ustawieniem.

8.10. Kosztem aplikacji jest jedna inferencja

Tryb `--bench` mierzy czas pojedynczej inferencji. Na maszynie odniesienia wynosi on 39 ms, a model pytany jest co 40 ms, wobec czego wątek inferencji pozostaje nasycony przez cały czas wybrzmiewania akordu; wszystkie pozostałe wątki — rysowanie, CQT, wywołanie zwrotne audio — dają łącznie poniżej 3%.

Ta sama binarka na tej samej maszynie pod systemem Windows raportuje 61 ms. Pozorna dziesięciokrotna różnica obciążenia pomiędzy systemami okazała się różnicą pomiędzy dwoma licznikami, nie pomiędzy dwiema kompilacjami: `top` podaje wartość w jednostkach jednego rdzenia, Menedżer zadań w skali całego procesora, wobec czego 100% rdzenia przy ośmiu rdzeniach to te same 12,5%.

8.11. Formuły oraz reguła surowsza niż w trybach dźwiękowych

Aplikacja losuje zbiór interwałów nad prymą — każdy podzbiór dwunastu funkcji chromatycznych zawierający prymę, łącznie 2048 — a ćwiczenie polega na odnalezieniu ich na gryfie i zagraniu w dowolnej kolejności. Funkcja raz zaliczona pozostaje zaliczona do końca rundy, co zmienia koszt zaliczenia fałszywego: w pozostałych trybach błędny odczyt opóźnia ćwiczenie, tutaj usuwa z niego funkcję bezpowrotnie.

Regułę zatem zmierzono, zamiast ją założyć. Na 49 dźwiękach rzeczywistego nagrania (`dist/latency_stats.py`):

reguła	zaliczenia fałszywe	dźwięki pominięte
cztery ścieżki naraz, jak w trybach dźwiękowych	110	0
sama jednoramkowa estymata CQT	33	0
to samo, bramkowane głowicą ataków	15	4

Ze 110 dziewięćdziesiąt dziewięć pochodziło z głowicy wysokości modelu — która odpowiada na pytanie „co brzmi", a struna wybrzmiewająca albo rezonująca współczująco brzmi, nie będąc zagraną. Formuły działają więc na samej estymacie jednoramkowej, z głosowaniem czterech z pięciu ostatnich ramek audio; bramki atakowej tu nie przyjęto, z powodu podanego w 8.12.

Ten sam tryb potrafi również postawić formułę na akordzie: jej pryma sadzana jest na jednym z dwunastu stopni akordu, po czym zliczane jest, ile z tego akordu zbiór pokrywa — wszystkie jego dźwięki poza prymą i czystą kwintą, bo tylko one ustalają, jaki to akord. Jest to arytmetyka na dwóch dwunastobitowych maskach i jest dokładna, co czyni ją wartą pokazania na ekranie obok funkcji.

8.12. Stan przeniesiony przez granicę

Model odpowiada o 0,77 s dźwięku. W chwili, gdy ćwiczenie przechodzi dalej — następny akord, następna runda, wejście w tryb — jego najświeższa odpowiedź dotyczy jeszcze tego, co było przed tą granicą. Aplikacja tę odpowiedź zachowywała i zaliczała pierwszy cel nowego akordu z wybrzmiewania poprzedniego.

Objaw zgłaszany był jako właściwość modelu: „przy pierwszym uruchomieniu było lepiej", „przedtem wykrywanie było szybsze". Nie był ani jednym, ani drugim. Pierwszy akord po starcie jest czysty, bo nie ma czego dziedziczyć; każdy następny zaczyna, trzymając poprzedni. Poprawka polega na porzuceniu tego, co usłyszano, przy każdej takiej granicy — wektora wysokości, poprzedniej ramki, odpowiedzi o uderzeniach, okna głosowania i ostatniego zaliczenia. Porzucenie nic nie kosztuje: następna ramka audio jest 16 ms dalej, a następna inferencja 40 ms.

Przez pewien czas głowica ataków była dostępna jako opcja — model mógł zaliczyć wyłącznie klasę, którą raportował również jako uderzoną. Po poprawieniu granicy przestała być potrzebna dla zgłoszonego objawu, a niosła własny koszt: w powyższym pomiarze zdejmowała 15 zaliczeń fałszywych ceną 4 dźwięków pominiętych zupełnie. Od 0.5.10 głowicy nie ma; „Zaliczaj tylko to, co uderzone” korzysta zamiast niej z detektora uderzeń (8.13).

8.13. Powtórzenia: która struna została uderzona

Klasa raz zaliczona może liczyć się ponownie wyłącznie wtedy, gdy od tamtej chwili **ta klasa** została uderzona — nie gdy uderzono jakąkolwiek strunę i nie gdy klasa jedynie nadal brzmi. Wszystko, czym aplikacja dysponowała, odpowiadało na jedno z tych dwóch łatwiejszych pytań: strumień zmian widma mówi, że uderzono strunę, i jest ślepy na to, którą; ucho i liczba głosów mówią, które klasy brzmią, i są ślepe na to, czy właśnie je uderzono. Do wersji 0.5.7 reguła była z nich składana, a harness z rozdziału 2 zmierzył, co przepuszczała: powtórzenie dozwolone, gdy dźwięk jedynie wybrzmiewał, 24 razy na 51.

Detektor uderzeń (5.4) odpowiada na tę koniunkcję wprost. Między nim a mechanizmem oceny stoją trzy elementy, każdy dodany, ponieważ wymagał go pomiar:

- **Wyrównanie poziomu.** Jego cechy nie są niezależne od poziomu, a cichy gracz wygląda jak słaby atak: na własnym nagraniu użytkownika, o 11 dB cichszym od nagrania, na którym go mierzono, znalazł 42 z 87 dźwięków. Powolne wzmocnienie, ustalające się w około cztery sekundy, sprowadza grę do poziomu znanego modelowi: 81 z 87.
- **Okres refrakcji 0,6 s na klasę.** Model odpala również na wybrzmiewających dźwiękach. W ćwiczeniach klasa wraca dopiero po zaliczeniu, 0,35 s pokazywania ukończonego zestawu i odpowiedzi gracza, więc nic rzeczywistego nie przepada.
- **Sprawdzenie energii przy ponownym odpaleniu klasy w ciągu 2 s.** Na trzech nagraniach użytkownika model odpalił klasę ponownie w ciągu 2 s 37 razy, w dwóch grupach bez niczego pomiędzy: 29 przy energii stałej lub malejącej — dźwięk zanika — oraz 8 przy jej skoku od 3,7 do 49 razy — struna uderzona ponownie. Takie odpalenie czeka 32 ms i liczy się tylko, jeśli energia wzrosła o jedną czwartą. Czekanie jest istotą rzeczy: rozstrzygane w chwili odpalenia, to samo sprawdzenie odrzucało rzeczywiste ponowne uderzenia, bo nowy dźwięk ledwie wszedł wtedy do okna.

zmierzone na 51 sklejonych dźwiękach	0.5.7	0.5.8
powtórzenie dozwolone, gdy dźwięk jedynie wybrzmiewa	24	**0**
uderzony ponownie po 0,8 s, zaliczony na czas	36, oraz 11 za wcześnie	**48**, żadnego za wcześnie
uderzony ponownie po 1,2 s, zaliczony na czas	35, oraz 14 za wcześnie	**50**, żadnego za wcześnie
dźwięk wybrzmiewający w kolejny akord, zaliczony tam	47	**0**

Ostatni wiersz to osobna reguła. To, co nadal brzmi, gdy ćwiczenie przechodzi do kolejnego akordu, liczy się tam jako już wykorzystane i potrzebuje własnego uderzenia — chyba że uderzono je, gdy ukończony zestaw był jeszcze pokazywany, co oznacza gracza sięgającego po następny akord wcześniej. Wymaganie uderzenia przy każdym pierwszym zaliczeniu zamknęłoby ten sam przeciek, lecz kosztuje 7 do 11 z 87 dźwięków, których detektor nie łapie; przenoszenie nie kosztuje nic tam, gdzie dźwięk zagrano po tym, jak o niego poproszono.

W trybie Interwały granie pojedynczo, stała kolejność i „tylko uderzone” są teraz domyślnie włączone — razem stanowią zmierzoną ścieżkę. Po wyłączeniu wszystkich trzech tryb staje się tym drugim, czym może być: rozstrzyga model i nic z nim nie polemizuje, łącznie z przenoszeniem — akordy i pojedyncze dźwięki akordowe grane dla przyjemności, nie jako test. Bez pliku detektora aplikacja nadal startuje i ocenia powtórzenia na starszych przesłankach, z zamkniętymi dwoma największymi przeciekami; ten tryb zastępczy przepuszcza 13 z 51 wybrzmiewających dźwięków.

9. Hipotezy zweryfikowane negatywnie

Rozdział dokumentuje przypadki, w których pomiar obalił wcześniej przyjęte założenie.

hipoteza	wynik pomiaru
Rozbieżność normalizacji (w ramce wobec globalnej) stanowi główne ograniczenie	różnica poniżej 1 pp; warstwa `InstanceNorm2d` ją kompensuje
Bariera harmoniczna — trzecia harmoniczna b3 przypada na b7, wobec czego min7 i min są nierozróżnialne	pomiar wykonano na segmentach z etykietami losowymi
Model nie odwzorowuje danych treningowych (niedouczenie)	kontrola TRAIN: 83,7% wobec 63,1% na walidacji, a więc przeuczenie
Dopasowanie szablonów przewyższy głowicę quality (B ≈ 75% wobec A = 63%)	B = 61,0%
Sufit prymy na poziomie 64% wynika z voicingów jazzowych bez prymy	artefakt nagrań solowych; w akompaniamencie 97%
Maskowanie prymy odblokuje jakość powyżej 73%	jakość pozostała na poziomie 72%
Chroma w dystrybuowanym pliku jest jednoelementowa, a więc nieprawidłowa	`cq_to_chroma` przy 24 binach na oktawę również przypisuje jedną wagę na bin; rozbieżność dotyczyła przesunięcia
Bez zmiennej `ORT_DYLIB_PATH` binarka wykorzysta bibliotekę systemową	`RUNPATH=$ORIGIN` z pliku `.cargo/config.toml` rozwiązywał to zagadnienie
Model pogorszył się w rozpoznawaniu pojedynczych dźwięków	na dźwiękach izolowanych przypisuje 0,96–0,99 właściwej klasie; na gamie jego okno 0,77 s zalicza dźwięk poprzedzający grany, w 79% okien
Akord zmniejszony septymowy nazwany od innego swojego dźwięku to inny akord	to te same cztery dźwięki: `Cdim7`, `Ebdim7`, `Gbdim7` i `Adim7` różnią się wyłącznie tym, który z nich model uzna za prymę, a to wynika z układu chwytu, nie z tego, co zagrano
Głowica ataków będzie lepszą bramką — jest najszybszą dostępną odpowiedzią	na nagraniu tak wyglądało: 202 ms wobec 676 ms. Zastosowana na żywo odrzucała znacznie więcej, niż wyłapywała, a w regule zaliczania wymieniła 18 zaliczeń fałszywych na 4 dźwięki pominięte
Tercja zaliczona przy granej prymie to piąta harmoniczna tej prymy	`--probe` na 364 oknach: zaliczenia fałszywe padają na +10 i +11 półtonów, czyli na dźwięk POPRZEDNI, wciąż obecny w oknie, a nie na harmoniczną
Jedna świeża odpowiedź głowicy ataków to okno zbyt krótkie, by uchwycić uderzenie	przy progu 0,02 odpowiedź utrzymuje się nad progiem przez medianę jednej sekundy po ataku, a żaden z 47 dźwięków nie został bez ramki, która by ją niosła — trzymaną w tym celu pamięć szesnastu ramek usunięto
Model uderzeń trenowany z rozrzutem poziomu ±6 dB nie zależy od głośności gracza	na nagraniu cichszym o 11 dB znalazł 42 z 87 dźwięków; po wyrównaniu poziomu przed modelem 81
Ponowne odpalenie klasy można odróżnić od wybrzmiewania po energii w tej chwili	model odpala około 26 ms po ataku, gdy w oknie dominuje jeszcze stary dźwięk; rozstrzygane wtedy sprawdzenie odrzucało rzeczywiste ponowne uderzenia. Rozstrzygane 32 ms później — rozdziela je

hipoteza	wynik pomiaru
Skok oktawy w oszacowaniu wysokości oznacza nowe szarpnięcie	50 z 58 fałszywych powtórzeń starszej reguły pochodziło z tej jednej gałęzi; obecnie wymaga ona ataku

Zależność jest jednoznaczna: **wyniki pomiarów potwierdzały się konsekwentnie, natomiast przewidywania formułowane przed pomiarem okazywały się błędne w sposób systematyczny.** Uzasadnia to przyjętą metodykę opartą na sondach.

---

10. Decyzje projektowe

10.1. Rozwiązania przyjęte

Generator wypisuje etykiety wprost. Żaden krok przetwarzania nie odtwarza informacji z sygnału.

Weryfikacja etykiet przez niezależny skrypt. Rozwiązanie nadmiarowe względem generatora, zastosowane celowo.

Podział zbioru po źródle. Obniża raportowane wskaźniki o kilkanaście punktów procentowych i jest uzasadniony.

Cztery głowice o rozdzielonych rolach. Tryby dźwiękowe opierają się na wektorze pitch, nie na nazwie akordu; czwarta głowica odpowiada za to, co uderzone, i jest odczytywana, zapisywana oraz udostępniona jako opcja, zamiast być wpięta w ocenianie.

Osobna sieć dla uderzeń. To, czy brzmiąca klasa została uderzona ponownie, rozstrzyga mała przyczynowa sieć pytana co 16 ms, a nie okno 0,77 s modelu rozpoznawania. Decyduje ona o powtórzeniach i o tym, co przechodzi przez granicę akordu; trener wytwarza ją w pliku, który aplikacja ładuje.

Dwa progi na oknie kontekstowym zamiast jednego. Model pytany jest od połowy okna, a jego nazwie akordu wierzy się od dziewięciu dziesiątych — jeden próg nie może obsłużyć zarazem trzymanego akordu i pojedynczego dźwięku.

Nic, co usłyszano przed granicą, jej nie przekracza. Zmiana akordu, koniec rundy i przełączenie trybu porzucają ostatnią odpowiedź modelu, ponieważ dotyczy ona tego, co było przed nimi.

Rzadka reprezentacja jądra CQT. Korzyść dwojaka: rozmiar pliku wag oraz czas przetwarzania w wątku audio.

Odrzucanie niezgodnych wag przez aplikację. Cicha akceptacja skutkowałaby programem funkcjonalnym, lecz błędnie klasyfikującym.

Napisy wkompilowane, bez biblioteki gettext. Przy kilkudziesięciu napisach zależność systemowa oraz katalogi `.mo` generują koszt przewyższający korzyść.

10.2. Rozwiązania odrzucone

Wyprowadzanie jakości z wektora pitch — zmierzone jako gorsze o 21 punktów procentowych od głowicy quality.

Agregacja czasowa jako sposób poprawy jakości — na kontrolowanej populacji daje około +1 pp. Błędy modelu są skorelowane w czasie: model nie wykazuje wahania pomiędzy oknami, lecz konsekwentnie i z wysoką pewnością wskazuje tę samą nieprawidłową odpowiedź.

Tryb taktowy (zapis przesuwający się w tempie, ocena zamiast bramki) — zaimplementowany, a następnie wycofany. Przyjęto założenie, że trenażer ma reagować dynamicznie.

Faza 3 treningu — zmierzona jako nieprzynosząca poprawy w trzech przebiegach.

10.3. Zagadnienia otwarte

- **Zbiór testowy z instrumentu docelowego.** Wszystkie wskaźniki dotyczą sześciu wykonawców zewnętrznych oraz dwóch renderów zbioru syntetycznego. Brak jest pomiaru na docelowym torze sygnału.
- **Zmiana `CTX_FRAMES` z 48 na 32** — przestała być wyborem swobodnym: wyeksportowany model ma wejście ustalone na 48 ramek, więc zmiana wymaga ponownego trenowania. Opóźnienie, któremu miała zaradzić, usunięto natomiast z tej ścieżki, na której miało znaczenie, oceniając pojedyncze dźwięki na jednej ramce CQT.
- **Estymator wysokości o krótszym oknie.** Autokorelacja w oknie rzędu 100 ms sprowadziłaby opóźnienie pojedynczego dźwięku poniżej okna FFT o długości 512 ms, które pozostaje ograniczeniem w szybkich przebiegach.
- **Zwiększenie ilości materiału z rzeczywistego instrumentu** — jedyny czynnik zdolny zmniejszyć różnicę 6,5 punktu procentowego.
- **Ciche dźwięki na bramce szumu.** Kwinta zagrana na poziomie −60 dBFS, na własnej bramce użytkownika, nie została zaliczona. Dźwignią jest bramka, nie reguła zaliczania, i żadnej reguły z tego powodu nie zmieniono.

---

11. Podsumowanie

W wyniku przeprowadzonych prac uzyskano model osiągający 92,4% trafień dokładnych na walidacji wyznaczonej z podziałem po źródle, wydany jako pakiety dystrybucyjne dla dwóch platform.

Zasadniczy przyrost dokładności nie wynikał ze zmian architektury, lecz z czterech ustaleń dotyczących danych:

1. połowa zbioru GuitarSet stanowi improwizację opisaną akordami akompaniamentu,
2. adnotacja `instructed` nie zawiera septym i błędnie klasyfikuje pięćset segmentów,
3. podział zbioru na poziomie segmentów wprowadza przeciek,
4. cele pitch należy wyznaczać z rzeczywistego wykonania, nie z zapisu.

Wymienione cztery zmiany przesunęły wskaźnik `Exact` z 44,8% na 92,4%. Żadna z nich nie dotyczyła struktury sieci.

Druga sieć potwierdziła ten wzorzec. Uczciwe powtórzenia dźwięków nie wynikły z większego modelu, lecz z precyzyjnie postawionego pytania — która klasa została uderzona, a nie która brzmi — zmierzonego na materiale o znanej odpowiedzi: zweryfikowanych dźwiękach jednego nagrania, sklejonych w sytuacje, które reguły muszą od siebie odróżnić.

Dokument opisuje stan na październik 2026, wersja 0.5.8. Repozytorium: https://github.com/greblus/solitito